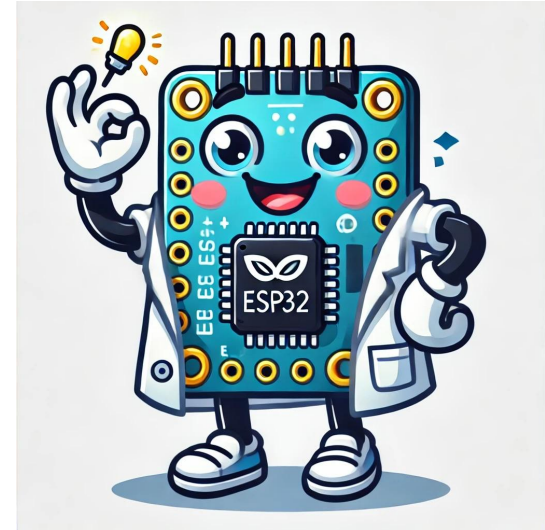


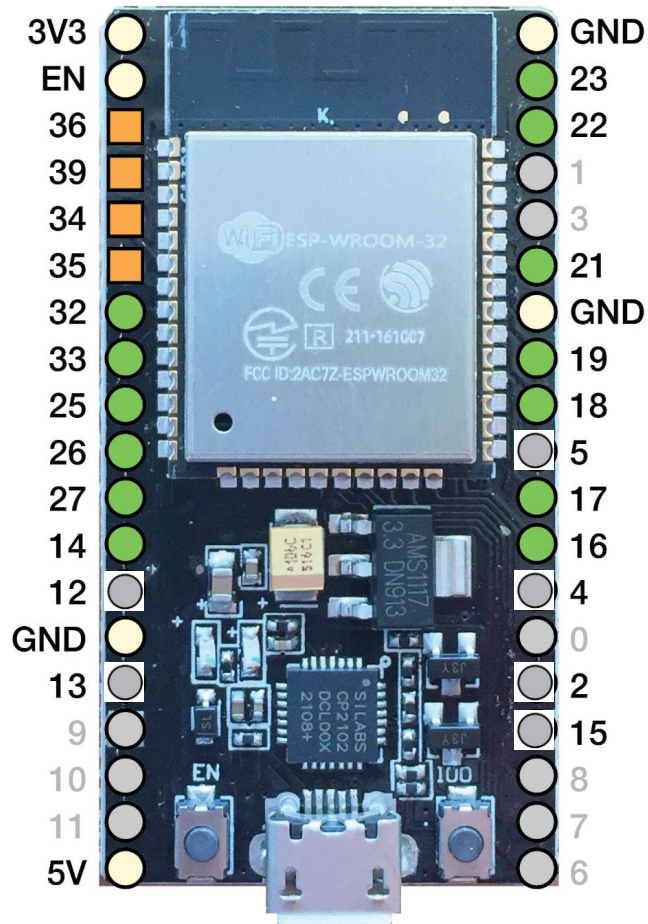


UNIVERSITÀ
degli STUDI
di CATANIA

ESP32: from Zero to Hero



davide.patti@dieei.unict.it



GPIO_NUMs

● Usable I/O pins

■ Input Only

● Non-GPIO

● Not Recommended

Problematic Pins

6, 7, 8, 9, 10, 11: Connected to the integrated SPI flash. Using them will crash your board instantly.

0, 2, 5, 12, 15: These are "Strapping Pins." Their voltage state at the exact moment the ESP32 powers on determines how the chip boots. If you have a sensor pulling one of these high or low during startup, your ESP32 may refuse to boot or fail to enter flashing mode.

4-13: Only if not using wifi while reading analog

Hands-on ESP32



<https://wokwi.com/esp32>
<https://esp32io.com/>

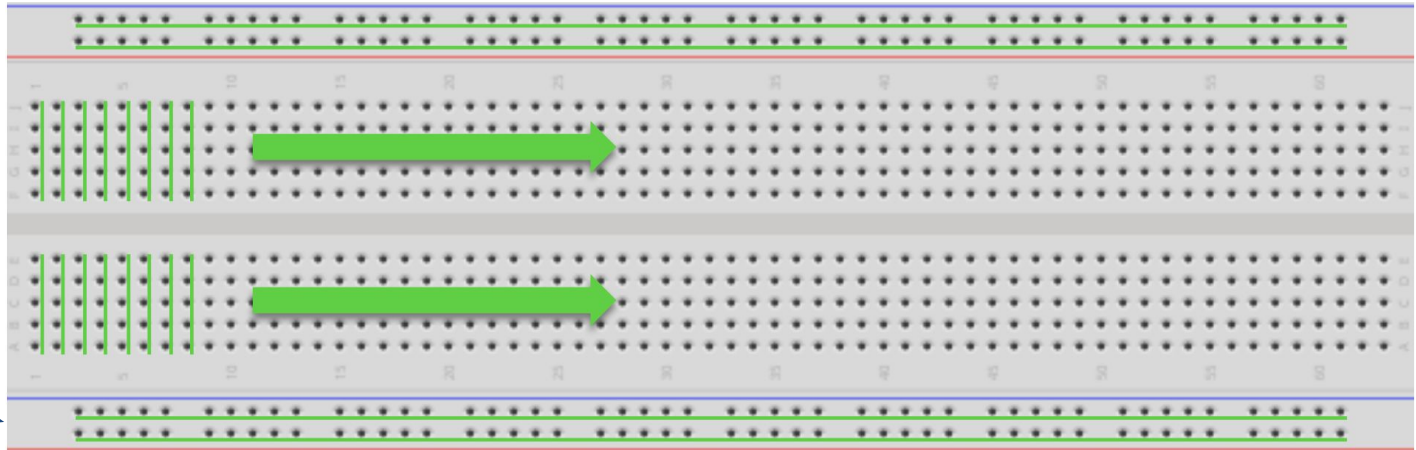
Most examples will be carried on the Wokwi simulator, but are directly applicable to the physical microcontroller with the Arduino IDE configured for running the ESP32

Suggested exercises will be proposed using this style

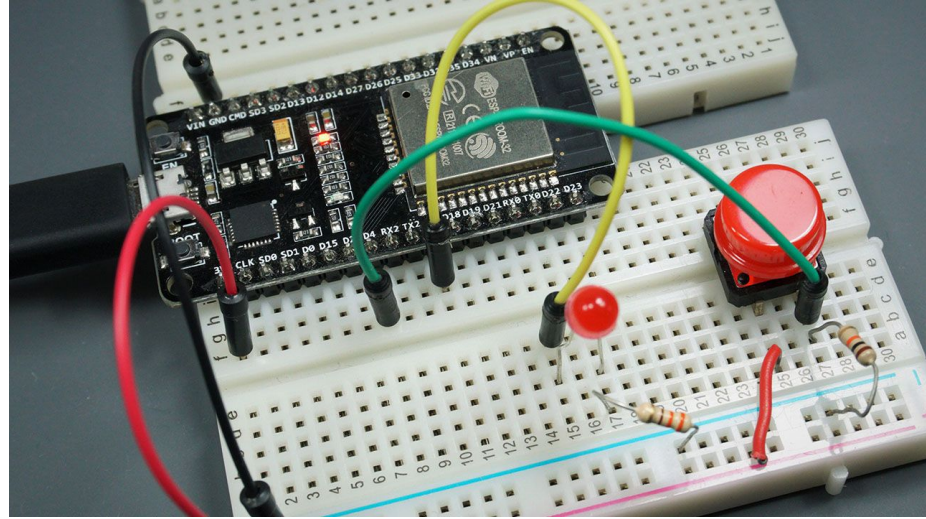
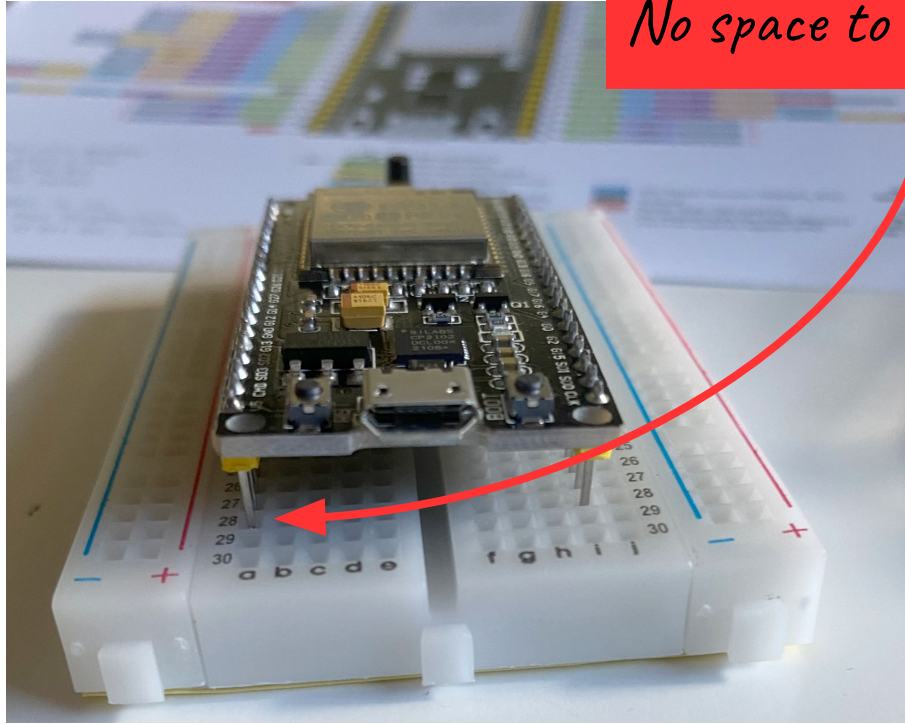
Breadboard

Horizontally connected lines, useful for voltage distribution (0, 3.3V)

- Useful for rapid prototyping
- Holes connected by green lines are short-circuited



No space to connect!

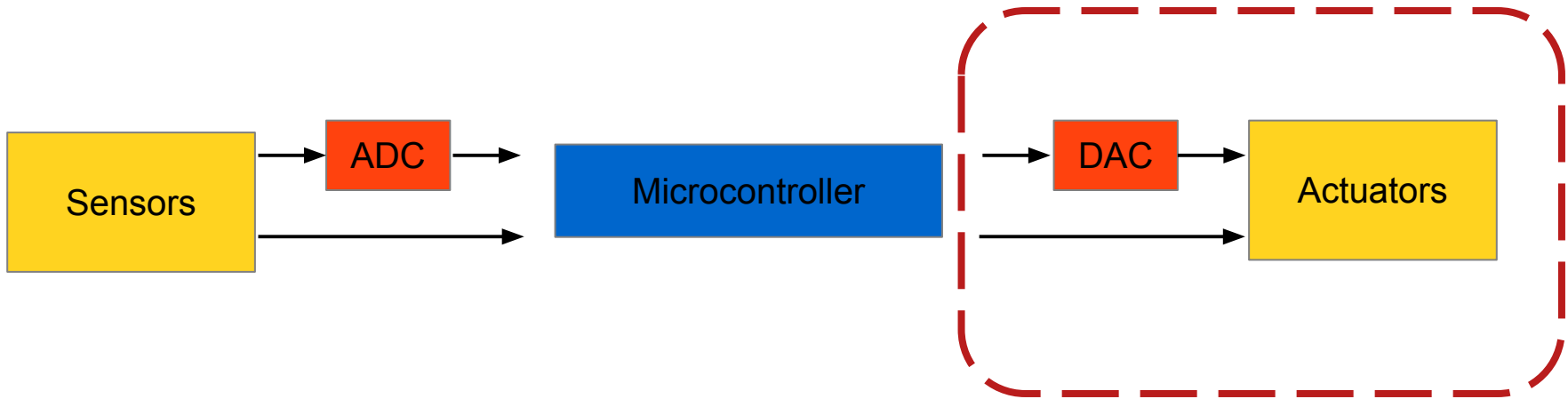


IMPORTANT: depending on the microcontroller size, you might need to arrange or split the breadboards!

- **Output (Actuators):**
 - Digital: LED on/off, Turning on/off a switch
 - Analog: LED light levels, fan speed, motors
- **Input (Sensors):**
 - Digital: button pressed/not pressed, Motion Detection
 - Analog: Temperature sensor, Light sensors

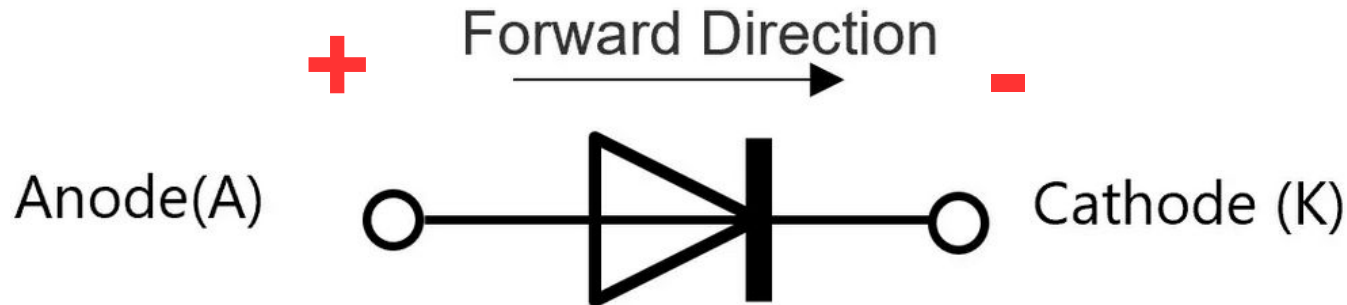
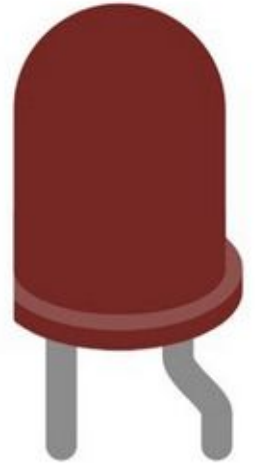


Let's focus on Output...



LED (Light Emitting Diode)

- A LED is an actuator: if powered appropriately, emits light as output
- When A voltage $>$ K voltage , becomes a closed circuit (current flows, emits light!)
- Otherwise, it's an open circuit (no current, no light)



Resistors & Current Limiting

Components are traversed by current flow generated by voltage differences (e.g., 0V to 3.3V).

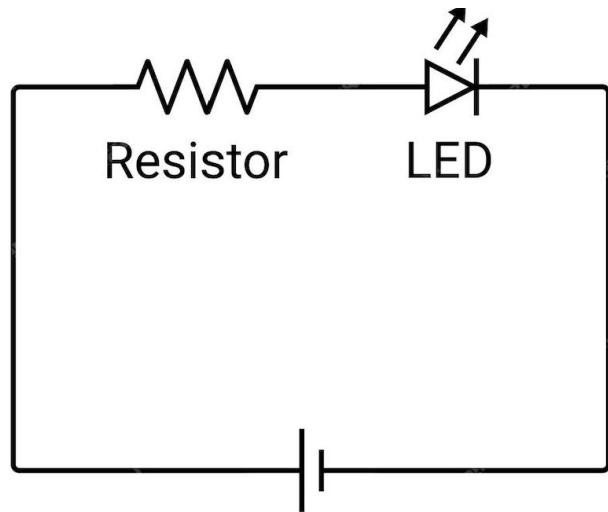
The Risk: No Resistor

Without a resistor, an LED acts as a short circuit ($R \approx 0$).

$I = V / 0 \rightarrow$ **Huge Current** $\rightarrow$ **LED Destroyed**

The Fix: LED Resistor Values

- **Too Big R:** Very low light
- **Too Small R:** Potential component damage
- **Typical:** ~220 Ohm is commonly used

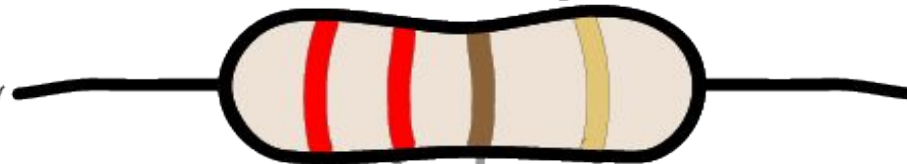


Standard Circuit Diagram: Resistor in series with LED

220 Ohm Resistor

2 2 x10 ±5%

sample resistor
with color bands



what the color
bands represent on
the resistor

	1st Digit	2nd Digit	Multiplier	Tolerance
Black	0	0	1	5% Gold
Brown	1	1	10	10% Silver
Red	2	2	100	
Orange	3	3	1,000	
Yellow	4	4	10,000	
Green	5	5	100,000	
Blue	6	6	1,000,000	
Purple	7	7		
Gray	8	8		
White	9	9		

Example: LED Blink

```
#include <Arduino.h>

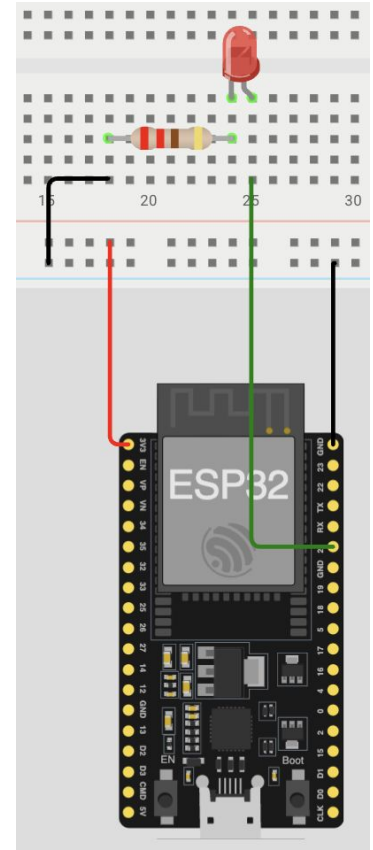
int ledPin = 21; // choose the pin for the LED

void setup() {
  // put your setup code here, to run once:
  pinMode(ledPin, OUTPUT); // declare the LED pin as an OUTPUT
  Serial.begin(115200); // start serial communication at 115200 baud rate
}

void loop() {

  Serial.println("Hello, World!"); // print "Hello, World!" to the serial monitor
  delay(1000); // wait for a second
  digitalWrite(ledPin, HIGH); // turn the LED on
  delay(1000); // wait for a second
  digitalWrite(ledPin, LOW); // turn the LED off
}
```

- *Add another LED that blinks in the opposite period*



Example: LED Fade

```
#include <Arduino.h>

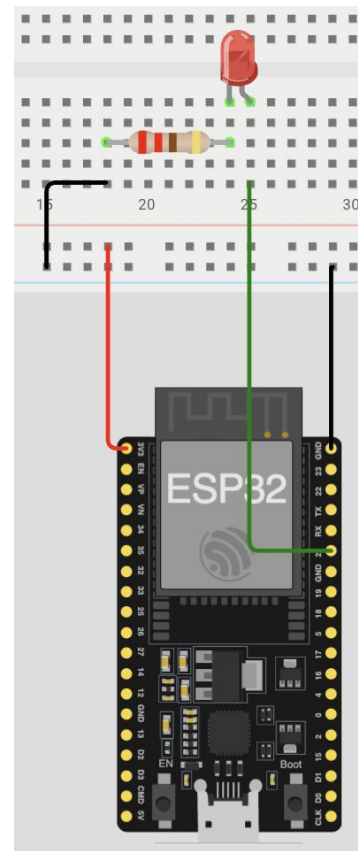
int ledPin = 21; // choose the pin for the LED
int lightLevel = 0; // variable to store the value coming from the sensor

void setup() {
  // put your setup code here, to run once:
  pinMode(ledPin, OUTPUT); // declare the LED pin as an OUTPUT
}

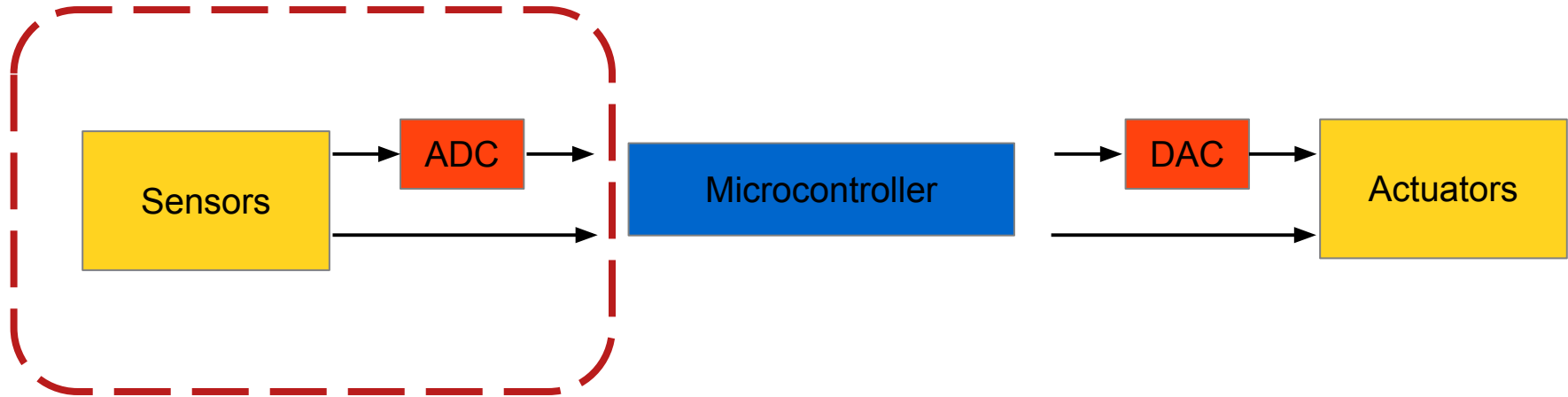
void loop() {
  for (lightLevel = 0; lightLevel <= 255; lightLevel++) { // fade in from min to max in increments of 5 points:
    analogWrite(ledPin, lightLevel); // change the LED brightness
    delay(10);
  }
  for (lightLevel = 255; lightLevel >= 0; lightLevel--) { // fade out from max to min in increments of 5 points:
    analogWrite(ledPin, lightLevel); // change the LED brightness
    delay(10);
  }
}
```

IMPORTANT: for simulator, the LED in diagram.json needs the gamma and fps attributes:

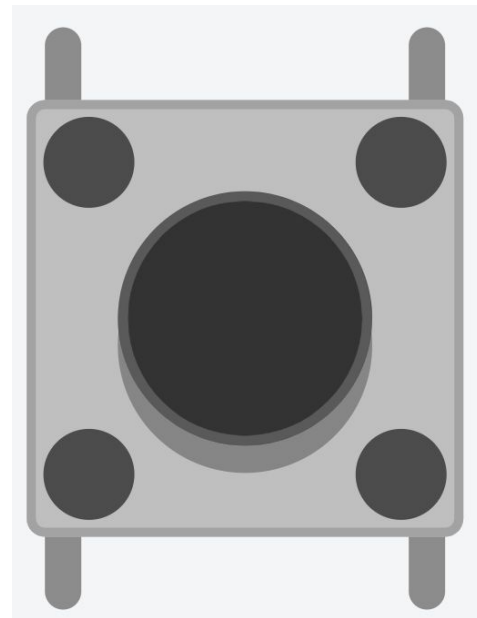
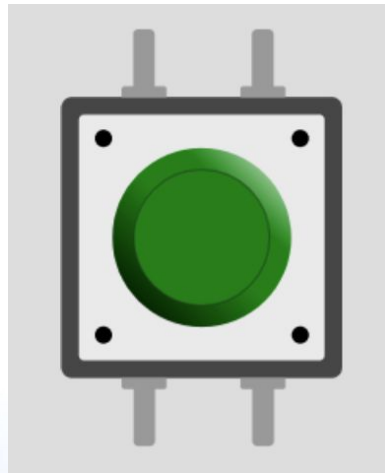
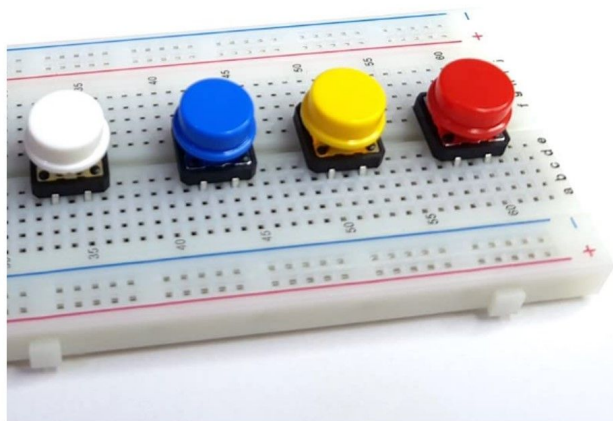
```
"attrs": {"color": "red", "gamma": "2.8", "fps": "80" }
```



Let's focus on Input...

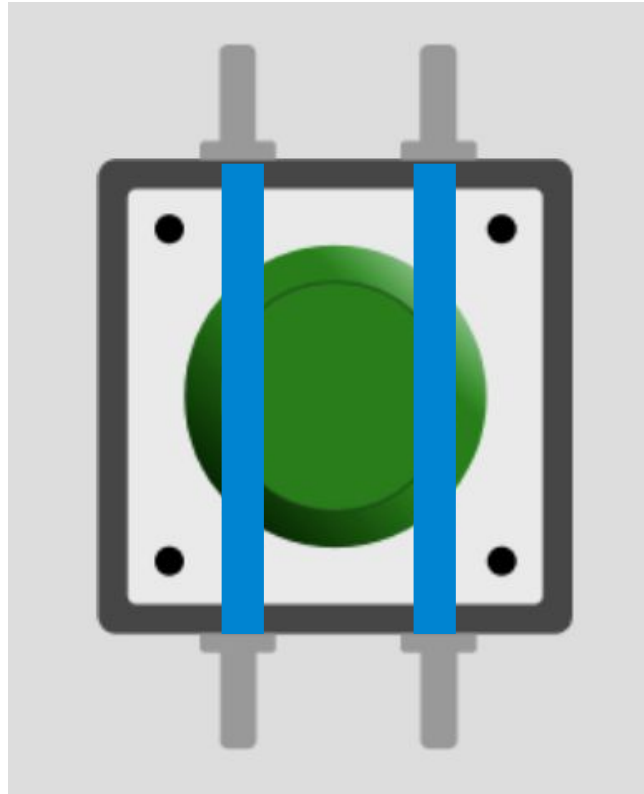


Button



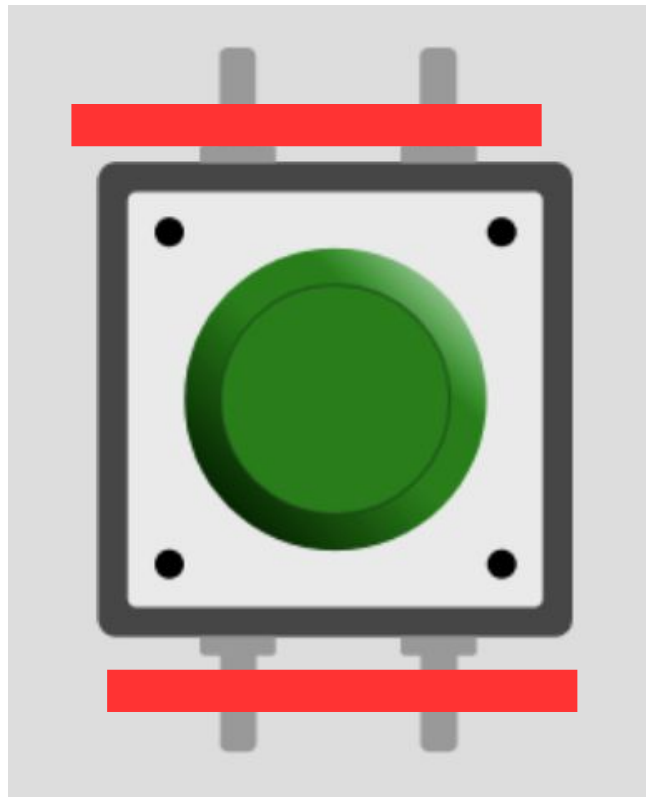
Button

Internally short
circuited, **ALWAYS**



Button

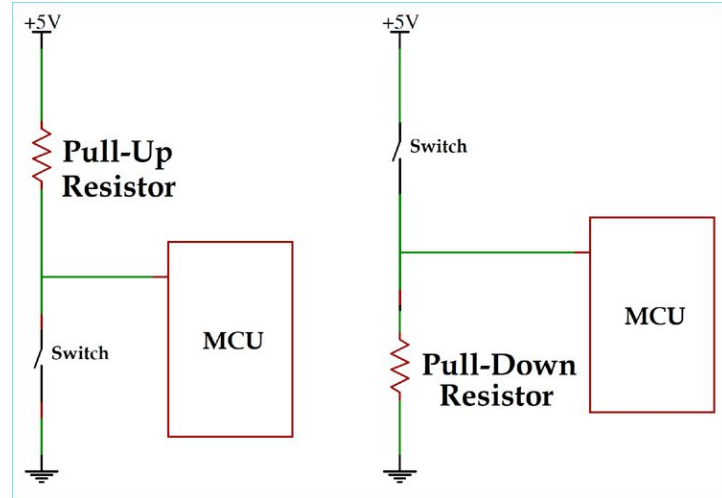
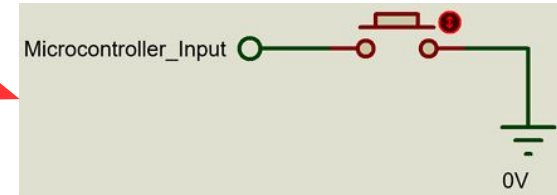
Push!



Reading a Button

What happens when button is open? (floating pin problem!)

- The button must switch between GND and power voltage (use digitalRead)
- You need a pull down/up resistor (you can choose how connect), to avoid short circuit!
- The resistance can be higher so that we consume less current (10k Ohm commonly)
- **IMPORTANT: always put a small delay (e.g. 100 ms) in the loop to avoid excessive frequency in reads**



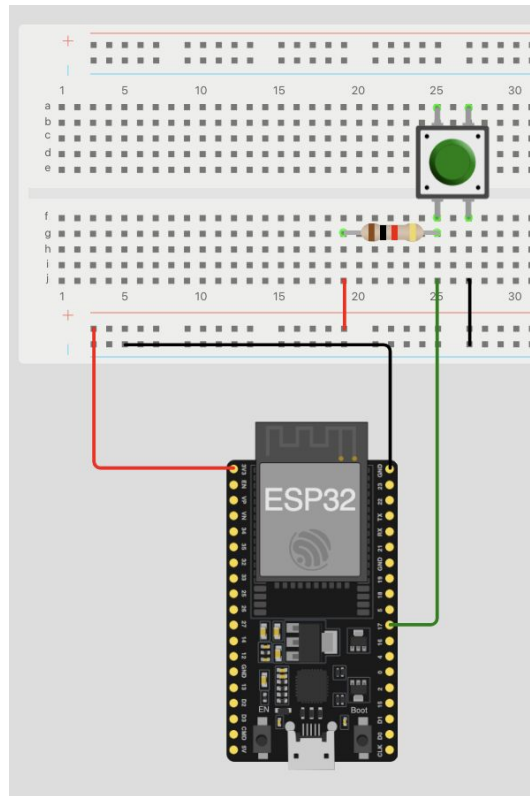
Example: Reading a button

```
#include <Arduino.h>

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  Serial.println("Hello, ESP32!");
  pinMode(17, INPUT);
}

void loop() {
  // put your main code here, to run repeatedly:
  delay(1000); // this speeds up the simulation

  if (digitalRead(17) == HIGH) {
    Serial.println("NOT PRESSED");
  }
  else
    Serial.println("PRESSED");
}
```

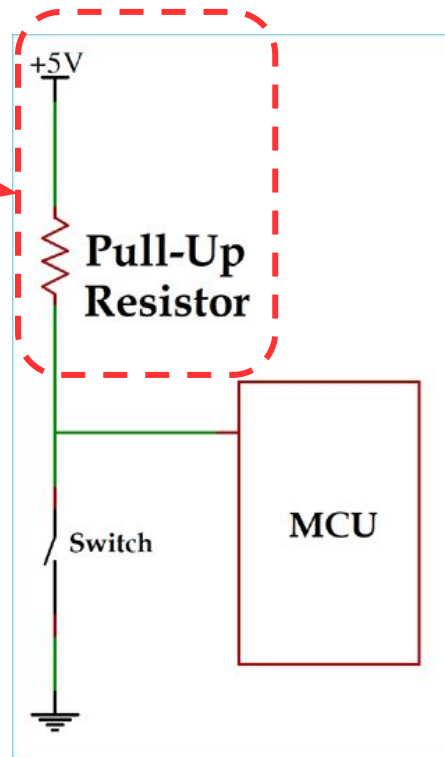


INPUT_PULLUP

- Considering the pullup version of the connection, inputs of ESP32 can optionally be configured to use an internal pullup resistor
- In this case, no external resistor is needed
- Be careful: not all microcontroller have such feature

```
Serial.begin(9600);  
// initialize the pushbutton pin as an pull-up  
pinMode(BUTTON_PIN, INPUT_PULLUP);  
}
```

Don't need it



Try to modify the previous example to use the input pullup feature

Example: Pedestrian Traffic Light System



Logic flow for an embedded system controlling vehicle traffic and pedestrian safety.

```

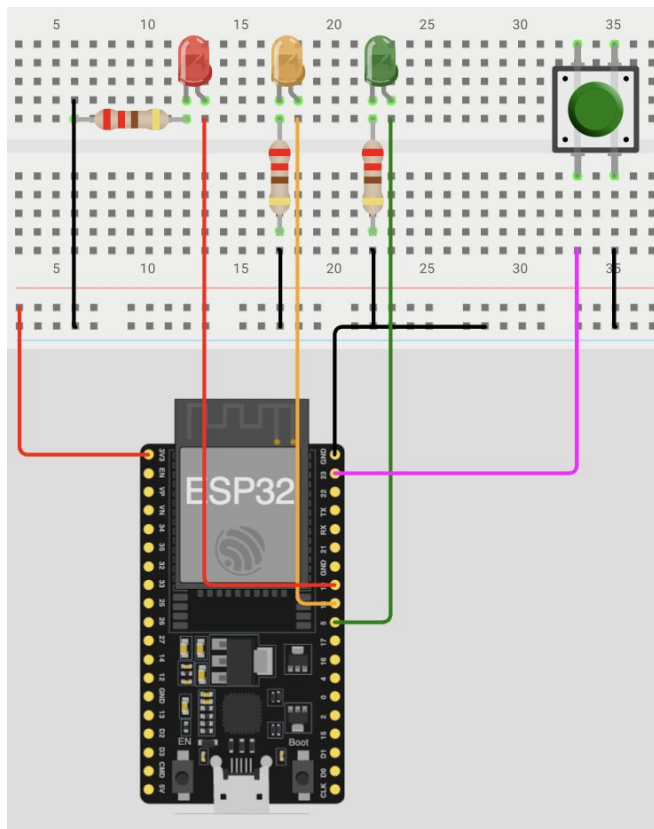
#include <Arduino.h>

int red_pin = 19;
int yellow_pin = 18;
int green_pin = 5;
int button_pin = 23;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  Serial.println("Hello, ESP32!");
  pinMode(red_pin, OUTPUT);
  pinMode(yellow_pin, OUTPUT);
  pinMode(green_pin, OUTPUT);
  pinMode(button_pin, INPUT_PULLUP);
}

void loop() {
  Serial.println("Nothing is happening...");
  if (digitalRead(button_pin)==LOW) {
    Serial.println("TADAAA!!! button pressed!");
    delay(2000);
    // green fixed, yellow stop blinkin
    digitalWrite(green_pin, HIGH);
    digitalWrite(yellow_pin, LOW);
    delay(2000);
    // yellow fixed
    Serial.println("cars... accelerate! (:");
    digitalWrite(green_pin, LOW);
    digitalWrite(yellow_pin, HIGH);
    delay(2000);
    Serial.println("Cars stop! people cross");
    digitalWrite(red_pin, HIGH);
    digitalWrite(yellow_pin, LOW);
    delay(5000);
    // otherwise it remains red + blinking yellow
    digitalWrite(red_pin, LOW);
  }
  delay(500);
  digitalWrite(yellow_pin, HIGH);
  | delay(500);
  digitalWrite(yellow_pin, LOW);
}

```



Advanced Button Press Detection



The Simple Approach

Previously, circuits only performed one task: checking the button. Delays weren't an issue as no other events were expected.



The Challenge

What if your project needs to monitor button states while simultaneously executing other complex computations?

Critical Bottlenecks in Polling:

- Where should the `digitalRead()` logic be placed for efficiency?
- Should check logic be repeated multiple times throughout the codebase?
- How to handle missed presses during `delay()` or heavy processing?

Moving beyond polling: exploring non-blocking code and hardware interrupts.

attachInterrupt(pin, ISR, mode)

Purpose: Attaches an interrupt to a GPIO pin, enabling the ESP32 to execute a specific function immediately when a hardware event occurs.

pin

GPIO pin number to monitor.
Works on almost any pin,
except for 6-11.

ISR

Interrupt Service Routine:
The function that will execute
when the trigger occurs.

mode

The type of signal change that triggers the interrupt (RISING, FALLING, CHANGE, LOW, HIGH).



Best Practices & ISR Management:

- Use **IRAM_ATTR** for ISRs to ensure they run from RAM (not flash) for speed.
- Keep ISRs **short and fast**; avoid delays or complex logic.
- Clean up with **detachInterrupt(pin)** when the interrupt is no longer needed.

Interrupts: Selecting the Mode

The choice of interrupt condition depends on the specific user interaction you want to capture.



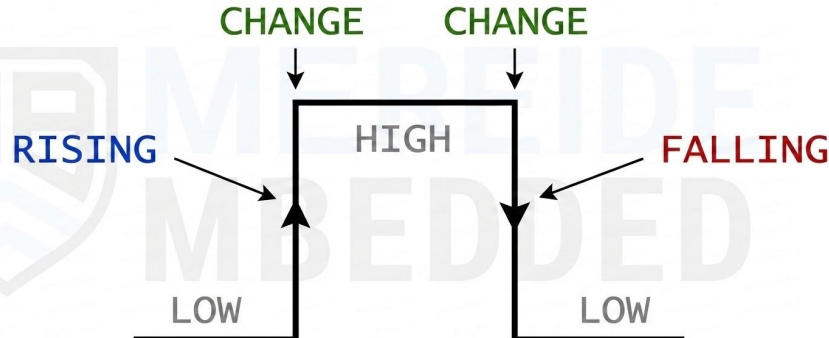
Case A: Detecting a Click

Triggers on a state transition.
Use **RISING** or **FALLING** to detect the moment the button is pressed or released.



Case B: Detecting a Hold

You cannot use both **RISING** and **FALLING** at the same moment.
Use the state change on the pin.



Use Case: Single Press Detection



The Objective

Implement a circuit where every click toggles the LED status.

(e.g., Click 1: OFF → ON, Click 2: ON → OFF)



Polling vs. Interrupt

Add a delay(5000) to compare:

- **Polling:** Periodic digitalRead() in the main loop.
- **Interrupt:** ISR associated with a pin FALLING event.

Implementation Note

While simple toggle circuits can be implemented without interrupts via polling, adding blocking delays (like the 5s delay above) makes polling-based detection nearly impossible, highlighting the efficiency of hardware interrupts.

```
#include <Arduino.h>

// Polling VS Interrupt: Uncomment to see what happens
/* IMPORTANT NOTES: in the simulator, remember to disable "bounce" checkbox in the button */

int pin_button = 23;
int counter = 0;

void button_pressed() {
    counter++;
    Serial.println("PRESSED!!");
}

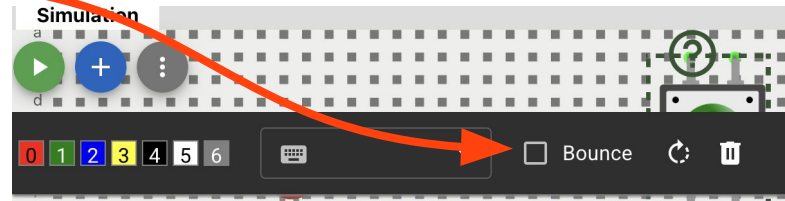
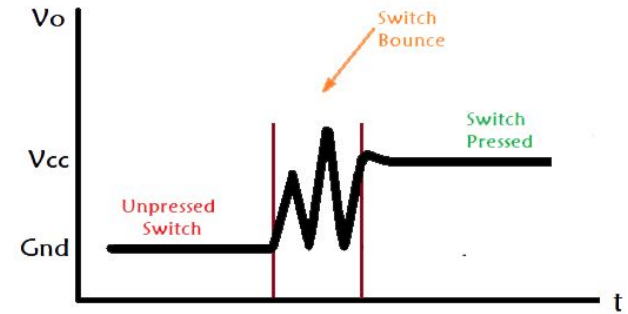
void setup() {
    Serial.begin(115200);
    pinMode(pin_button, INPUT_PULLUP);

    // *** INTERRUPT: uncomment to you attach a condition falling to the pin of the button
    attachInterrupt(pin_button, button_pressed, FALLING);
}

void loop() {
    // *** POLLING: uncomment to periodically check if button is pressed
    /*
    if (digitalRead(pin_button) == LOW) {
        Serial.println("Ehi, button pressed!!!");
        counter++;
        Serial.println(counter);
    }
    */
    Serial.println("Some other stuff begins to execute...");
    delay(5000);
    Serial.println("... and after a few seconds is finished!");
}
```

Button Bouncing Problem

- When the two wires of a button connect together, there is a transitory phase where the signal oscillates (for about 50ms)
- In simple circuits (not using interrupts) is not usually a problem, because of the delay present in the loop, used to avoid excessively frequent read/writes
- But when using interrupts, the handling function can be called multiple times at each “bounce”
- When simulating, you can **disable** this realistic effect (in the reality, you have to deal with it!)



Solving the Button Bouncing

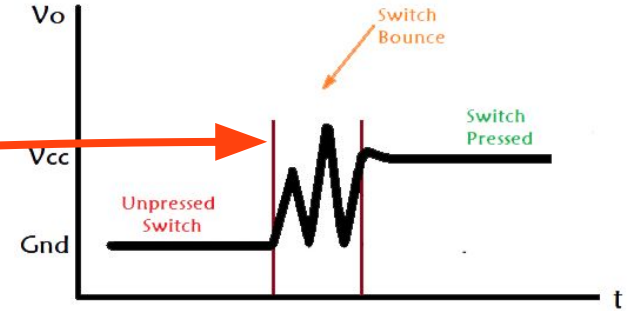
```
#define BUTTON_PIN 16
#define LED_PIN 14

volatile bool led_status = false;
unsigned long lastInterruptTime = 0;
const int debounceDelay = 50;

void IRAM_ATTR handleButtonPress() {
    unsigned long currentTime = millis();
    if (currentTime - lastInterruptTime > debounceDelay) {
        led_status = !led_status;
        digitalWrite(LED_PIN, led_status);
        Serial.println("Button Pressed!");
        lastInterruptTime = currentTime;
    }
}

void setup() {
    pinMode(BUTTON_PIN, INPUT);
    pinMode(LED_PIN, OUTPUT);
    attachInterrupt(BUTTON_PIN, handleButtonPress, FALLING);
    Serial.begin(9600);
}

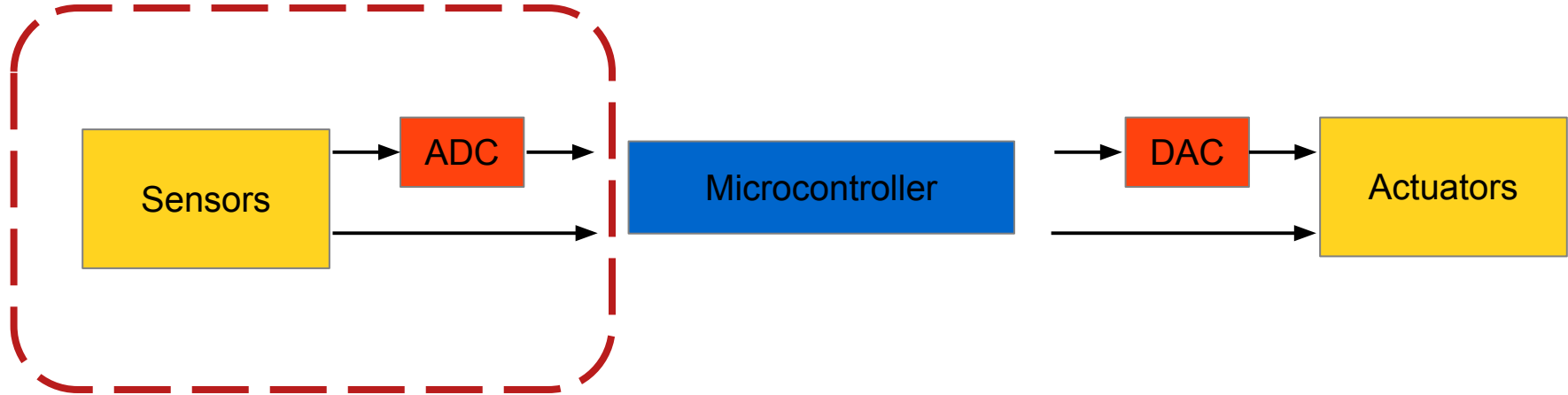
void loop() {
    delay(100);
    // other stuff happening here...
}
```



- Even if interrupt function has been called, do not execute its code if not enough time has passed since the last time it was executed
- 50 ms is usually a good choice

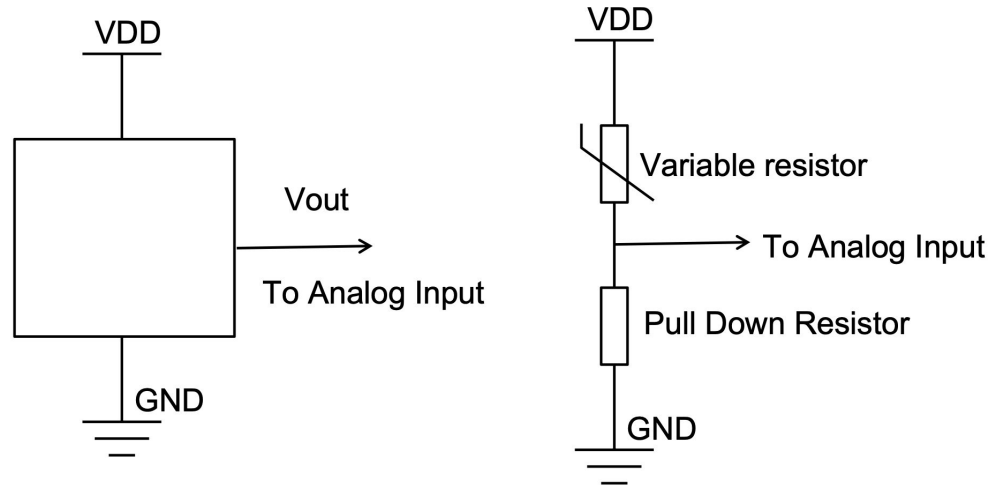
Implement a circuit that only while you are
keeping pressed it keeps the LED light on...
Which condition should use ?

- **Output (Actuators):**
 - Digital: LED on/off, Turning on/off a switch
 - Analog: LED light levels, fan speed, motors
- **Input (Sensors):**
 - Digital: button pressed/not pressed, Motion Detection
 - **Analog: Temperature sensor, Light sensors**



Sensors Pin Configurations: 3 vs. 2-Pin

Feature	3+ Pin Sensors (Active)	2-Pin Sensors (Passive)
Internal Circuitry	Yes (amplifiers, converters)	No (raw resistance/voltage change)
Power Pins	VDD, GND	N/A
Signal Output	Processed signal (analog/digital)	Raw voltage or resistance
Examples	LM35, DHT11, IR sensors, TSL2561	LDR, Thermistors (NTC, PTC), Force-sensitive resistors (FSR)
Advantages	Easier to interface, stable signal, calibrated output	Lower cost, flexible circuit integration, requires external processing



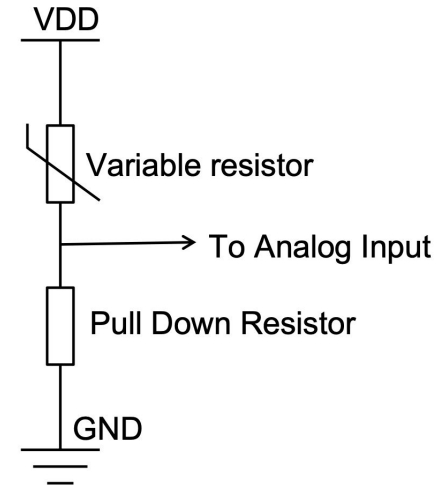
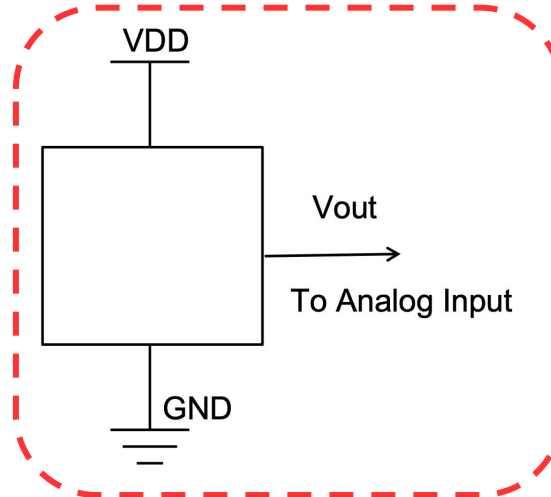
Sensors Pin Configurations: 3 vs. 2-Pin

Feature	3+ Pin Sensors (Active)	2-Pin Sensors (Passive)
Internal Circuitry	Yes (amplifiers, converters)	No (raw resistance/voltage change)
Power Pins	VDD, GND	N/A
Signal Output	Processed signal (analog/digital)	Raw voltage or resistance
Examples	LM35, DHT11, IR sensors, TSL2561	LDR, Thermistors (NTC, PTC), Force-sensitive resistors (FSR)
Advantages	Easier to interface, stable signal, calibrated output	Lower cost, flexible circuit integration, requires external processing

Internal Circuitry: These sensors often have built-in signal conditioning (e.g., amplification, filtering, or analog-to-digital conversion).

3 Pins:

- **VDD (Power):** Supplies voltage to the sensor.
- **GND (Ground):** Common reference point.
- **Vout (Signal Output):** Provides an already processed signal.



3+ Pin Sensors (Active)

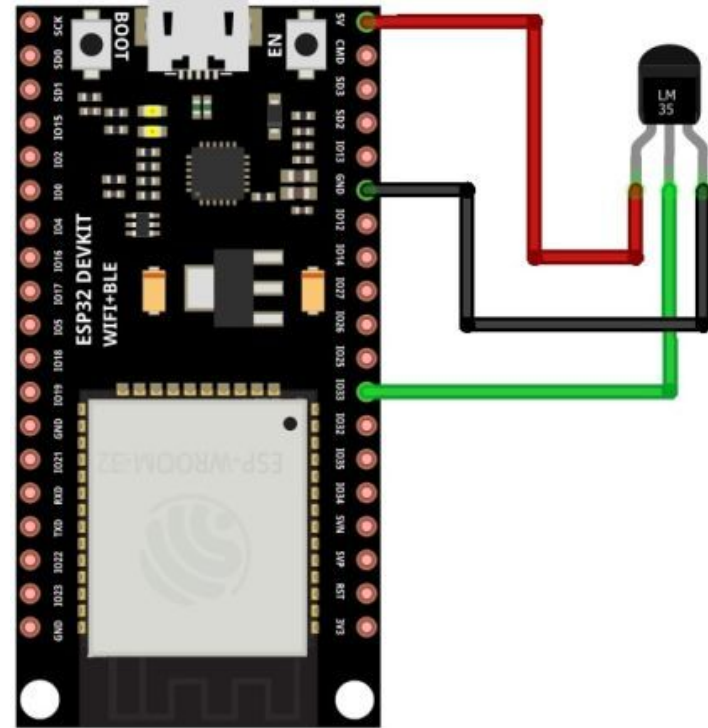
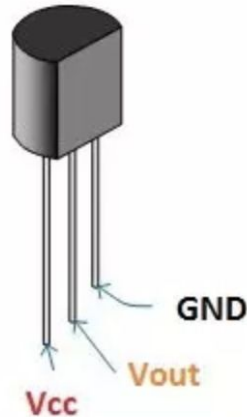
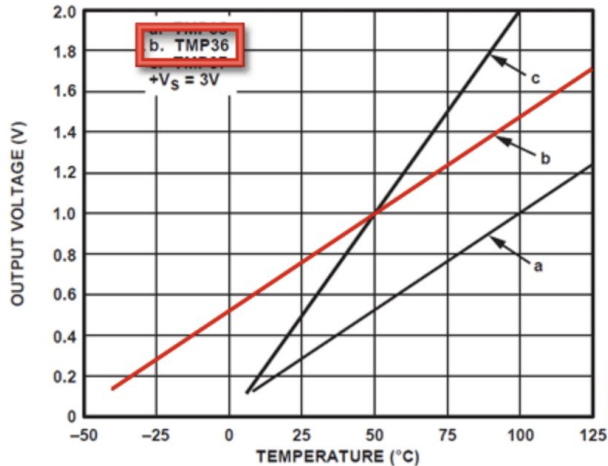
1. **LM35 (Temperature Sensor)** – Provides a linear analog output proportional to temperature (°C).
2. **DHT11/DHT22 (Humidity & Temperature Sensor)** – Digital output with built-in calibration.
3. **TSL2561 (Light Sensor)** – Measures ambient light with I2C communication.
4. **HC-SR04 (Ultrasonic Distance Sensor)** – Uses echo timing to measure distance digitally.
5. **MPU6050 (Accelerometer & Gyroscope)** – Measures acceleration and angular velocity via I2C.

2-Pin Sensors (Passive)

1. **LDR (Light-Dependent Resistor)** – Changes resistance based on light intensity.
2. **NTC/PTC Thermistors (Temperature Sensors)** – Resistance varies with temperature.
3. **FSR (Force-Sensitive Resistor)** – Resistance decreases when pressure is applied.
4. **Piezoelectric Sensor** – Generates voltage when subjected to mechanical stress.
5. **Microphone (Electret Condenser Type)** – Converts sound waves into analog voltage.

LM35 and TMP36 Analog Temperature Sensors

- ESP32 ADC is 12-bit (0-4095), so we scale it to 3.3V.
- LM35 provides a **direct temperature conversion** (mV to °C).
- TMP36 requires subtracting **0.5V** from the voltage before converting.
- Both sensors work **without external libraries**.



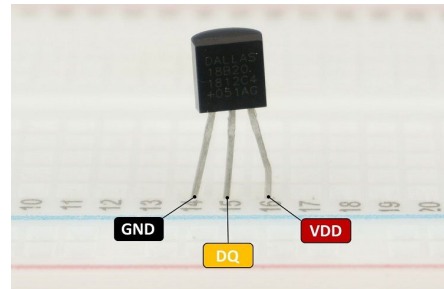
```
#define TEMP_SENSOR_PIN 34 // Connect to LM35 or TMP36 Vout
```

```
void setup() {  
    Serial.begin(115200);  
}  
  
void loop() {  
    int rawValue = analogRead(TEMP_SENSOR_PIN); // Read analog value (0-4095)  
    float voltage = (rawValue / 4095.0) * 3.3; // Convert to voltage (ESP32 ADC: 0-3.3V)  
  
    // Convert voltage to temperature  
    float temperatureC_LM35 = voltage * 100.0; // LM35: 10mV/°C  
    float temperatureC_TMP36 = (voltage - 0.5) * 100.0; // TMP36: Offset of 0.5V  
  
    Serial.print("LM35 Temperature: ");  
    Serial.print(temperatureC_LM35);  
    Serial.print(" °C | TMP36 Temperature: ");  
    Serial.print(temperatureC_TMP36);  
    Serial.println(" °C");  
  
    delay(1000);  
}
```

DS18B20: Digital Temperature Sensor

The DS18B20 is a digital temperature sensor that differs from classic analog sensors like the TMP36 or LM35

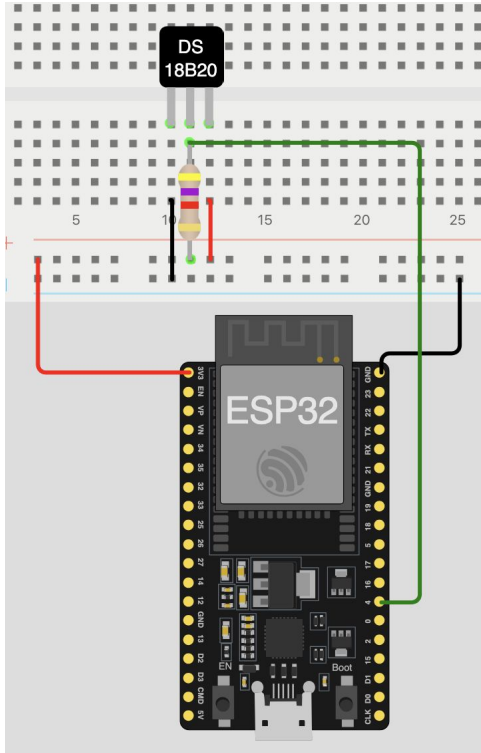
Digital Output (1-Wire Protocol): Unlike the TMP36 or LM35, which provide an analog voltage proportional to temperature, the DS18B20 communicates digitally via a 1-Wire interface, reducing noise issues and allowing multiple sensors on a single data line.



Recommended Libraries for Arduino IDE

1. **OneWire Library** – Handles low-level 1-Wire communication.
 - Install: `OneWire` by Jim Studt, Paul Stoffregen
2. **DallasTemperature Library** – Simplifies interaction with the DS18B20.
 - Install: `DallasTemperature` by Miles Burton

Notice: a pullup resistor (typically 4.7k) is required, because the 1 wire protocol output works by driving the signal down



```
#include <OneWire.h>
#include <DallasTemperature.h>

#define ONE_WIRE_BUS 4 // Use GPIO4 or any other available pin

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

void setup() {
    Serial.begin(115200);
    sensors.begin();
}

void loop() {
    sensors.requestTemperatures();
    float temperatureC = sensors.getTempCByIndex(0);
    Serial.print("Temperature: ");
    Serial.print(temperatureC);
    Serial.println(" °C");
    delay(1000);
}
```



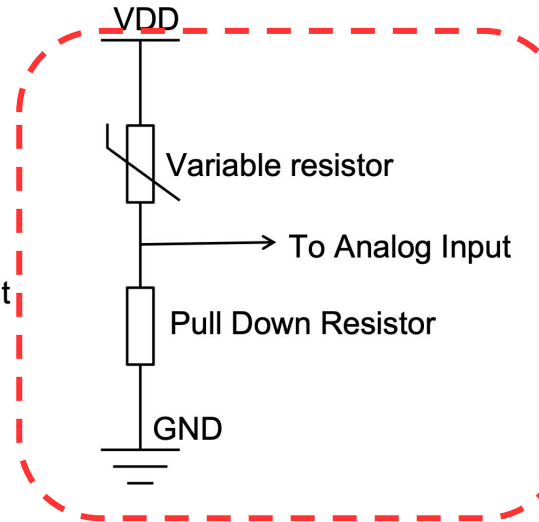
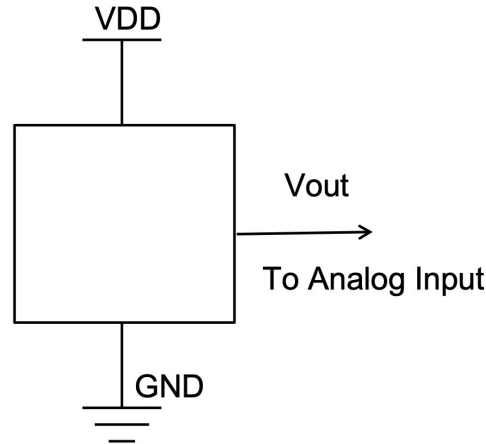
Sensors Pin Configurations: 3 vs. 2-Pin

Feature	3+ Pin Sensors (Active)	2-Pin Sensors (Passive)
Internal Circuitry	Yes (amplifiers, converters)	No (raw resistance/voltage change)
Power Pins	VDD, GND	N/A
Signal Output	Processed signal (analog/digital)	Raw voltage or resistance
Examples	LM35, DHT11, IR sensors, TSL2561	LDR, Thermistors (NTC, PTC), Force-sensitive resistors (FSR)
Advantages	Easier to interface, stable signal, calibrated output	Lower cost, flexible circuit integration, requires external processing

No Internal Circuitry: These sensors work as simple resistive or voltage-dividing components.

Configuration: Usually connected as a voltage divider with a pull-up or pull-down resistor.

Examples: LDR (Light-dependent resistor), Thermistors (NTC, PTC), Force-sensitive resistors (FSR)



Light Photoresistor (2 Pins)

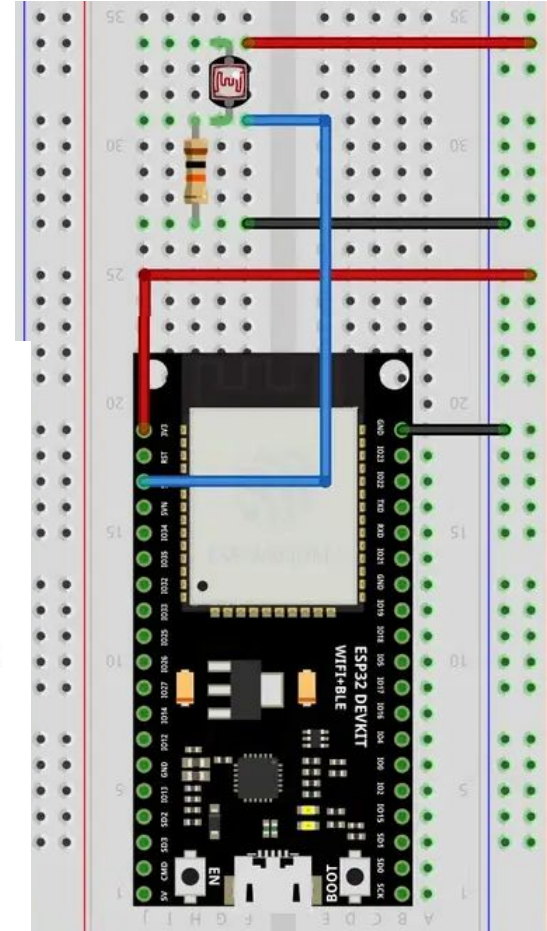
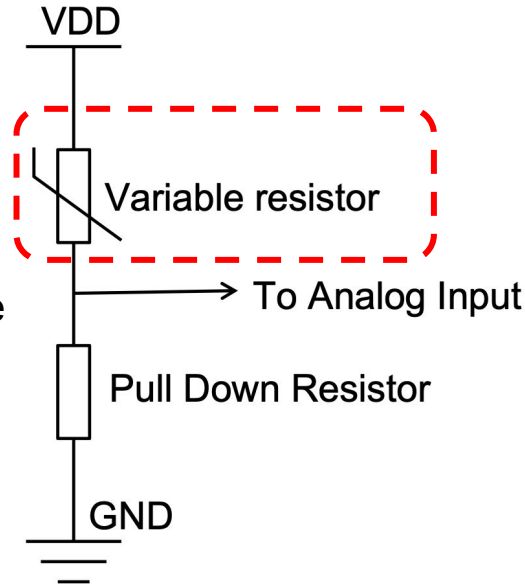
The sensor and the pull-down resistor form a voltage divider, where the output voltage (to the ESP32 analog input) is:

$$V_{out} = V_{cc} \times \frac{R_{\text{sensor}}}{R_{\text{sensor}} + R_{\text{pull-down}}}$$



Key points:

- These sensors don't have an internal circuitry that provides a processed signal on some output pin
- They just get "the effect" of the physical magnitude measured by changing their physical properties (the resistance)
- bigger the pulldown resistor-> less sensitivity to the variation of the other
- Generally some kilo Ohms it's ok



- **Calibration (First 5 Seconds):**

- Reads sensor values continuously.
- Stores minimum and maximum detected values.
- Helps adapt to different lighting conditions.

- **After Calibration:**

- Maps sensor readings to a percentage (0-100%).
- Displays light level in a readable format.

```
#define LDR_PIN 34 // LDR connected to GPIO34 (ADC)

int minLight = 4095; // Max possible ADC value
int maxLight = 0;    // Min possible ADC value

void setup() {
  Serial.begin(115200);
  Serial.println("Calibrating light sensor...");

  // Calibration phase (5 seconds)
  unsigned long startTime = millis();
  while (millis() - startTime < 5000) {
    int rawValue = analogRead(LDR_PIN);
    if (rawValue < minLight) minLight = rawValue;
    if (rawValue > maxLight) maxLight = rawValue;
    delay(50); // Small delay to sample different light conditions
  }

  Serial.println("Calibration completed!");
  Serial.print("Min Light Value: "); Serial.println(minLight);
  Serial.print("Max Light Value: "); Serial.println(maxLight);
}

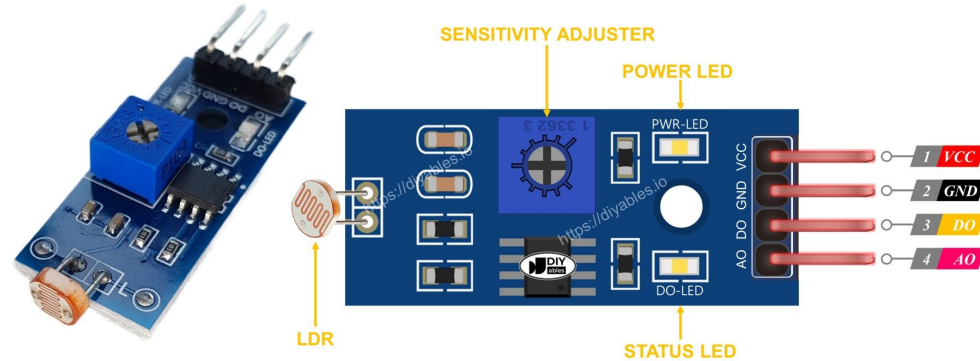
void loop() {
  int rawValue = analogRead(LDR_PIN);

  // Map the raw ADC value to a scale of 0 to 100
  int lightLevel = map(rawValue, minLight, maxLight, 0, 100);
  lightLevel = constrain(lightLevel, 0, 100); // Ensure within bounds

  Serial.print("Light Level: ");
  Serial.print(lightLevel);
  Serial.println(" %");

  delay(500); // Update every 500ms
}
```

Sometimes a multi-pin version of the sensor can be provided, where the core sensor component is integrated in a small circuit to make it easier to use

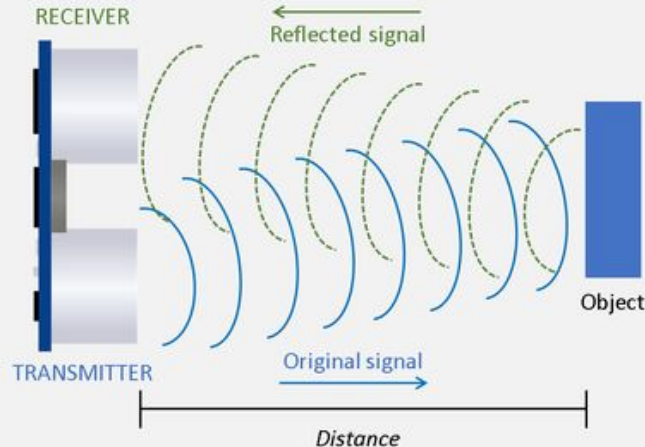
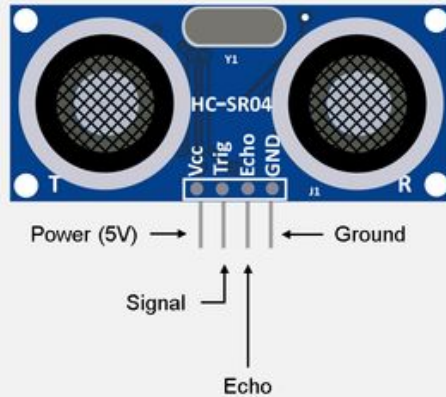


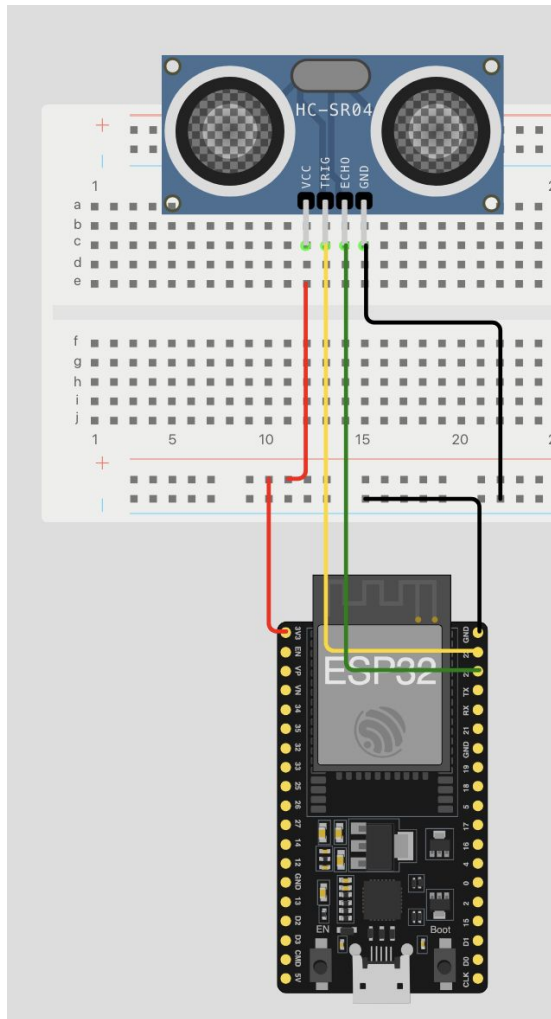
The LDR light sensor module has four pins:

- ◆ **VCC pin:** Connect this pin to the power source (between 3.3V to 5V).
- ◆ **GND pin:** Connect this pin to the ground (0V).
- ◆ **DO pin:** This is a digital output pin. It gives a **HIGH** signal when it's dark and **LOW** when it's light. You can adjust the threshold between dark and light using a built-in potentiometer.
- ◆ **AO pin:** This is an analog output pin. The value decreases as the light gets brighter and increases as the light gets darker.

HC-SR04 Ultrasonic Sensor

- The **TRIG** pin is used to start the measure (by using a **10 μ s pulse**.)
- The **HC-SR04 emits an ultrasonic wave** and puts signal HIGH on ECHO
- The wave **bounces back from an object** and is received by the **ECHO pin**, setting the pin LOW
- The **HIGH level duration of ECHO** is measured by the microcontroller.
- Notice: all the timing work is done by the sensor. Instead, using `millis()` to track begin and return of the sound would be inaccurate for such short timings





```
#define TRIG_PIN 23 // ESP32 pin GPIO23 connected to  
#define ECHO_PIN 22 // ESP32 pin GPIO22 connected to
```

```
float duration_us, distance_cm;
```

```
void setup() {  
    // begin serial port  
    Serial.begin(9600);  
  
    // configure the trigger pin to output mode  
    pinMode(TRIG_PIN, OUTPUT);  
    // configure the echo pin to input mode  
    pinMode(ECHO_PIN, INPUT);  
}  
  
void loop() {  
    // generate 10-microsecond pulse to TRIG pin  
    digitalWrite(TRIG_PIN, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(TRIG_PIN, LOW);  
  
    // measure duration of pulse from ECHO pin  
    duration_us = pulseIn(ECHO_PIN, HIGH);  
  
    // calculate the distance  
    distance_cm = 0.017 * duration_us;  
  
    // print the value to Serial Monitor  
    Serial.print("distance: ");  
    Serial.print(distance_cm);  
    Serial.println(" cm");  
  
    delay(500);  
}
```


1 PASSIVE RESISTIVE SENSORS (2 pins)

Example



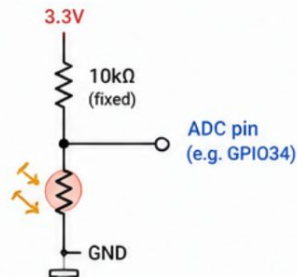
LDR / Photoresistor

Typical pins

2 pins



Wiring example (to ESP32)

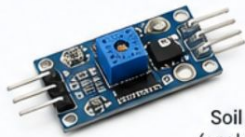


What the ESP32 reads



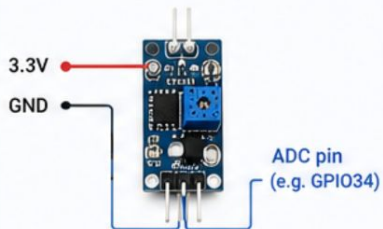
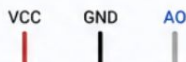
Sensor resistance changes. Use a voltage divider and read with `analogRead()`.

2 ANALOG-OUTPUT MODULES (3 pins)



Soil moisture (analog module)

3 pins

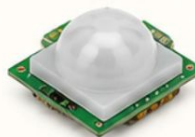


Module outputs a voltage proportional to the measured quantity. Read with `analogRead()`.

3 DIGITAL-OUTPUT SENSORS (2-4 pins)



Pushbutton



PIR sensor

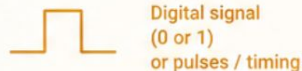
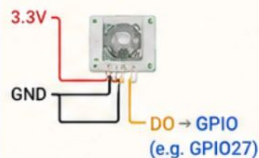
2 - 4 pins



Pushbutton (2 pins)

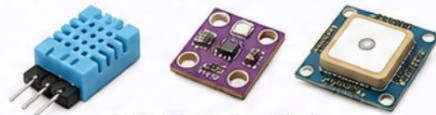


PIR sensor (3 pins)



Outputs logic levels or pulses. Use `digitalRead()`, interrupts, `pulseIn()` for timing/frequency.

4 PROTOCOL-BASED / SMART MODULES (3-6+ pins)



DHT22 (temp/humidity)
BME280 (env. sensor)
GPS module

3 - 6+ pins



I²C (e.g. BME280)



UART (e.g. GPS)



1011010... Digital data via protocol (I²C, SPI, UART, 1-Wire, etc.)

Communicate using a protocol. Use libraries or serial communication to get the measurements.